



BITS Pilani
Pilani Campus

Securing and Testing scalable services

Akanksha Bharadwaj
Asst. Professor, CSIS Department



BITS Pilani
Pilani Campus



SE ZG583, Scalable Services

Lecture No. 11

Agenda



Securing and Testing scalable services

- Securing code and repositories
- Using Authentication and Authorization
- Testing

Securing code and repository



- Your code is only as secure as the systems used to create it.
- Version control, peer review and built-in auditing are some of the advantages which come with using a code repository.
- If proper attention is paid to security measures, the benefits of using a repository far outweigh the risks.

Measures for securing the code and repository



- Choose a repository you trust
- Consider the exposure of your repository
- Protect access credentials
- Access to the repository should be revoked when no longer required, or in the event of compromise
- Encryption
- Code Signing
- Ensure your code is backed up
- Auditing and Logging

What is Authentication?



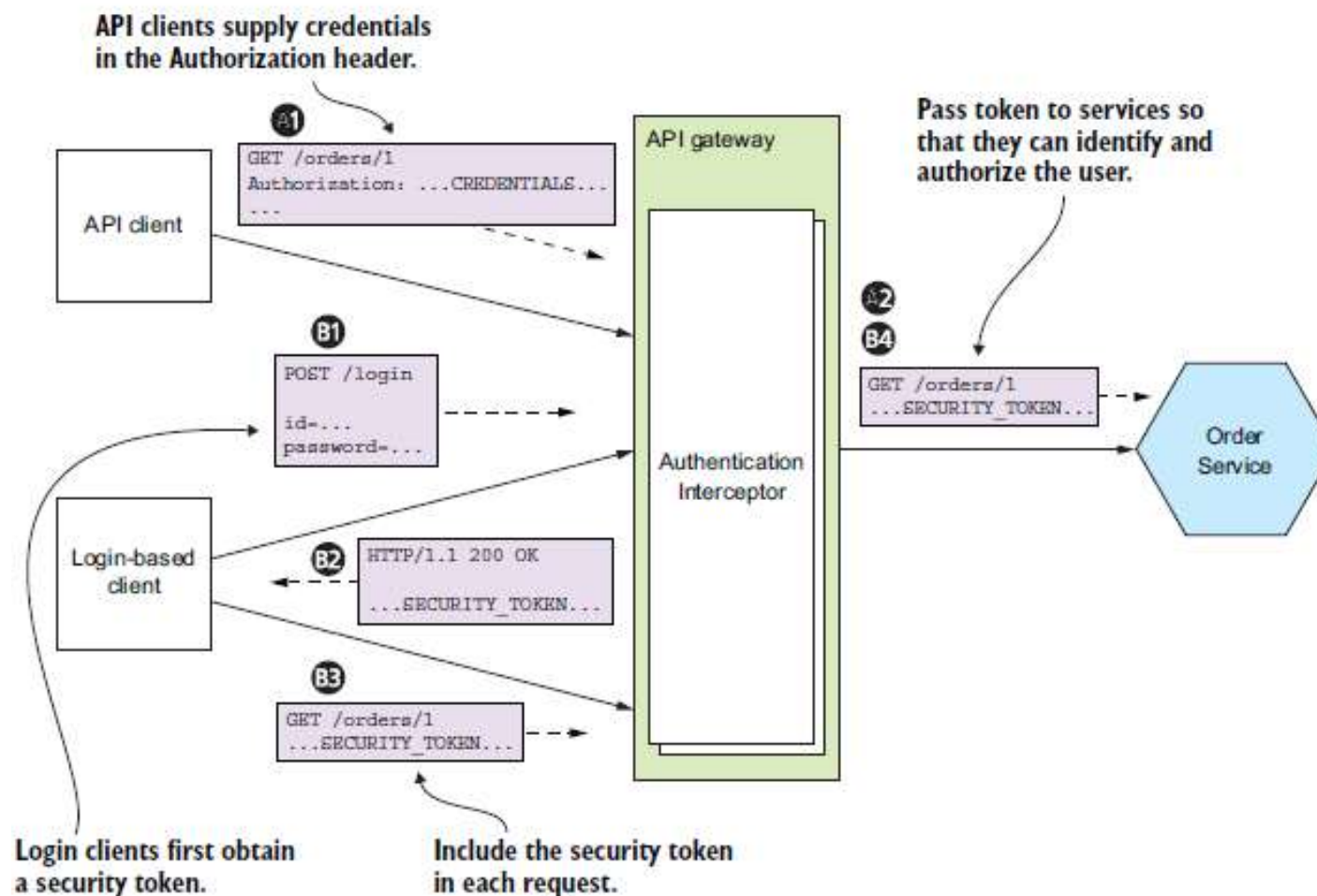
- Verifying the identity of the application or human that's attempting to access the application.

Authentication implementation options



- One option is for the individual services to authenticate the user.
- Other approach is for the API gateway to authenticate a request before forwarding it to the services.

Authentication in API Gateway





The sequence of events for API clients is as follows:

- A client makes a request containing credentials.
- The API gateway authenticates the credentials
- It creates a security token, and passes that to the service or services.



The sequence of events for login-based clients is as follows:

- A client makes a login request containing credentials.
- The API gateway returns a security token.
- The client includes the security token in requests that invoke operations.
- The API gateway validates the security token and forwards it to the service or services.

Authorization



- Verifying that the principal is allowed to perform the requested operation on the specified data.
- Applications often use a combination of rolebased security and access control lists (ACLs).

Authorization in API Gateway



- If a user isn't allowed to access a particular path, the API gateway can reject the request before forwarding it on to the service.
- We can implement authorization in the API gateway using a security framework, such as Spring Security.

Key aspects and benefits of implementing authorization in an API gateway



- Centralized Access Control
- Scalability and Performance
- Flexibility
- Logging and Auditing
- Protection Against Attacks

Explore: What are the possible drawbacks of this approach?

Authorization in Services



- The other place to implement authorization is in the services.
- A service can implement role-based authorization for URLs and for service methods.
- It can also implement ACLs to manage access to aggregates.

Key aspects and benefits of implementing authorization within services



- Fine-Grained Control
 - Decentralization
 - Service Autonomy
 - Scalability
 - Flexibility
 - Security Isolation
-
- **Explore:** What are the possible drawbacks of this approach?

Which approach is better?



The preferred approach depends on the specific requirements and characteristics of the system:

- **Centralized Control:** If uniformity and centralized management of access control policies are paramount, implementing authorization in an API gateway might be preferred.
- **Fine-Grained Control:** For systems with diverse access control requirements or where services have unique authorization needs, implementing authorization within services may be more suitable.
- **Scalability and Performance:** If scalability and performance are critical concerns, especially in high-traffic environments, offloading authorization to an API gateway might be preferable.
- **Security Isolation:** If security isolation is a priority and potential single points of failure are a concern, embedding authorization within services could be favored.
- In many cases, a hybrid approach may be appropriate, combining elements of both centralized and decentralized authorization mechanisms to achieve a balance between control, scalability, performance, and security.

What is OAuth?



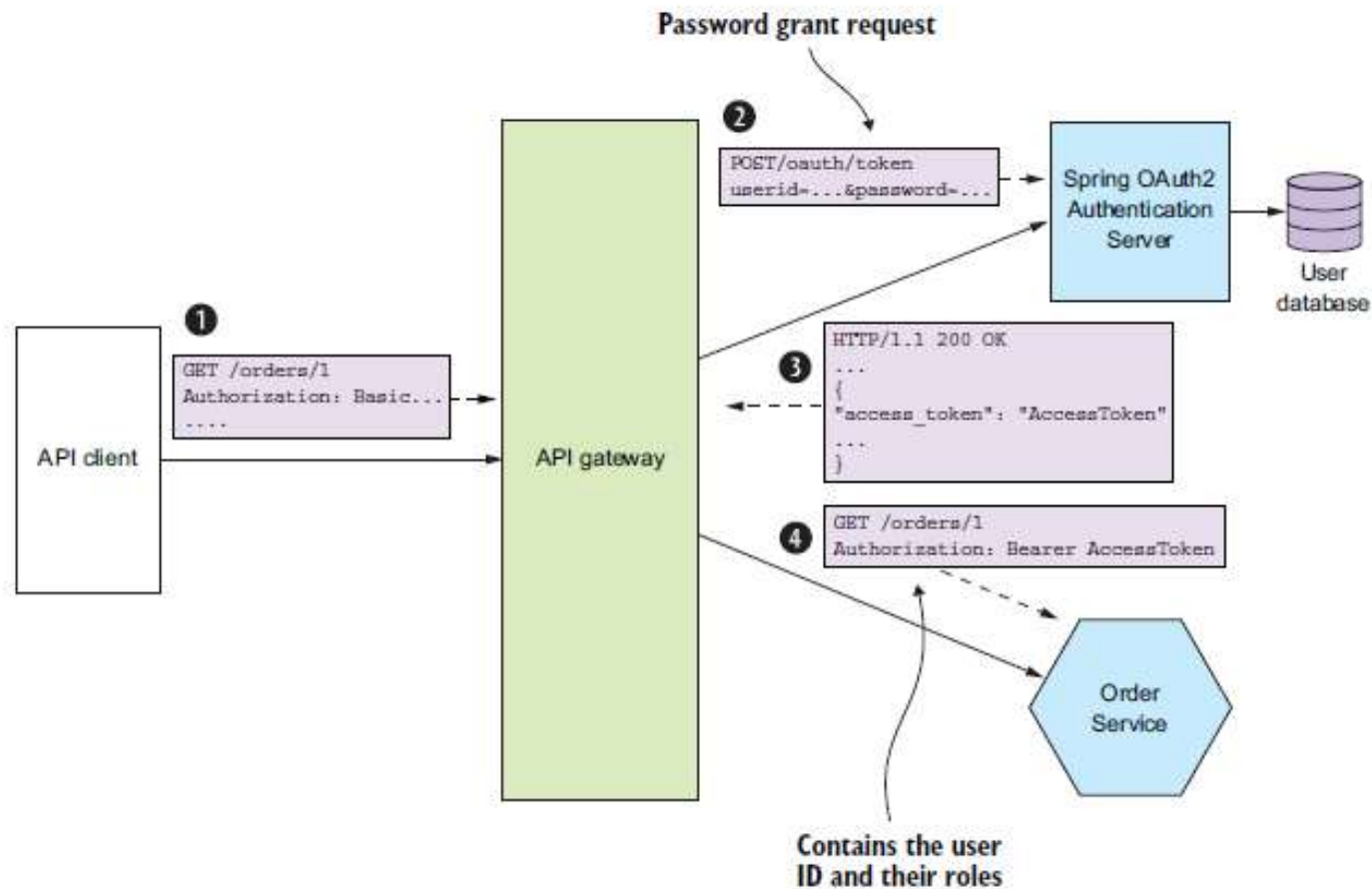
- OAuth is an open-standard authorization protocol or framework
- The simplest example of OAuth is when you go to log onto a website and it offers one or more opportunities to log on using another website's/service's logon.
- You then click on the button linked to the other website, the other website authenticates you

Terms used in OAuth



- Authorization Server
- Access Token
- Refresh Token
- Resource Server
- Client

How OAuth works for Microservices



Latest Trends in Securing Microservices



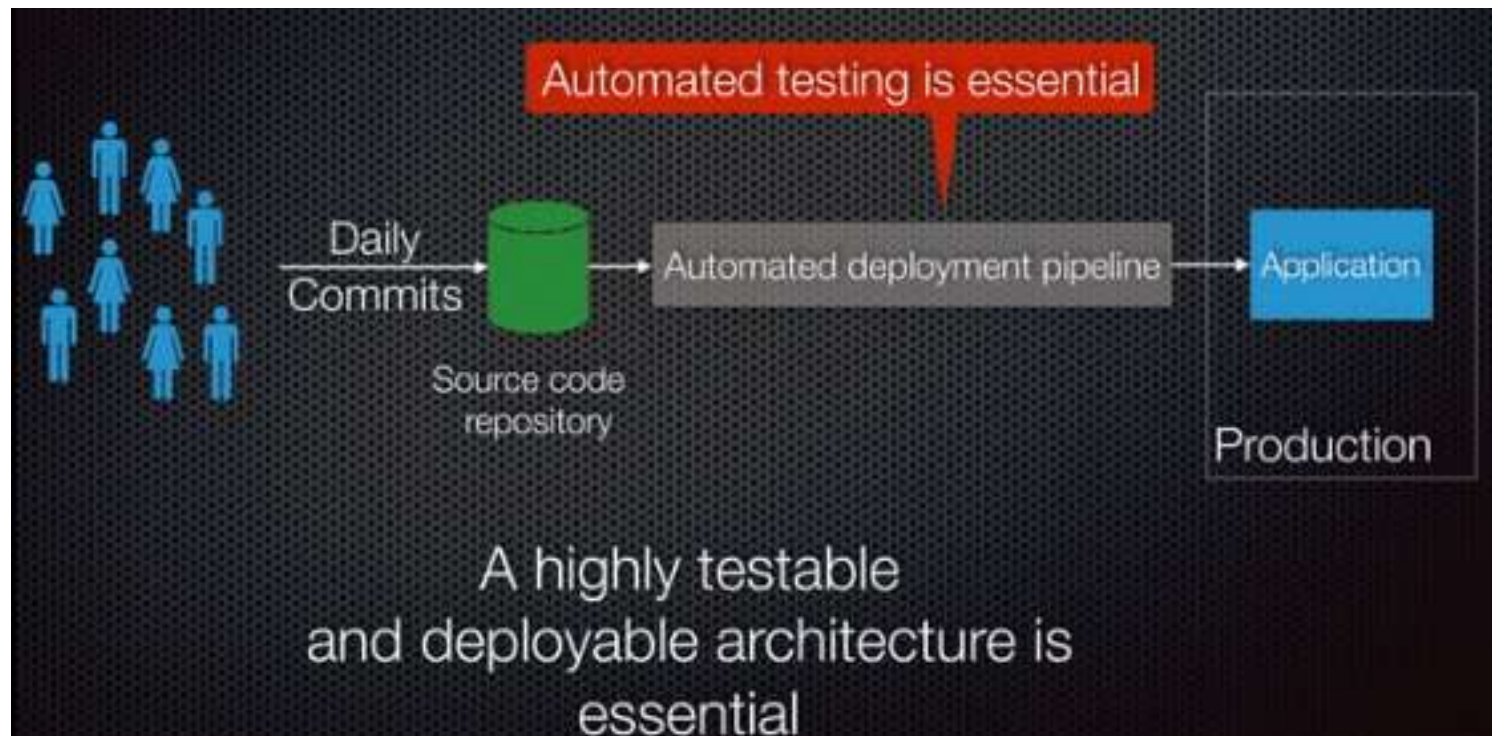
- Zero Trust Architecture Adoption for Microservices Security
- Runtime Application Self-Protection (RASP) in Microservices



BITS Pilani
Pilani Campus

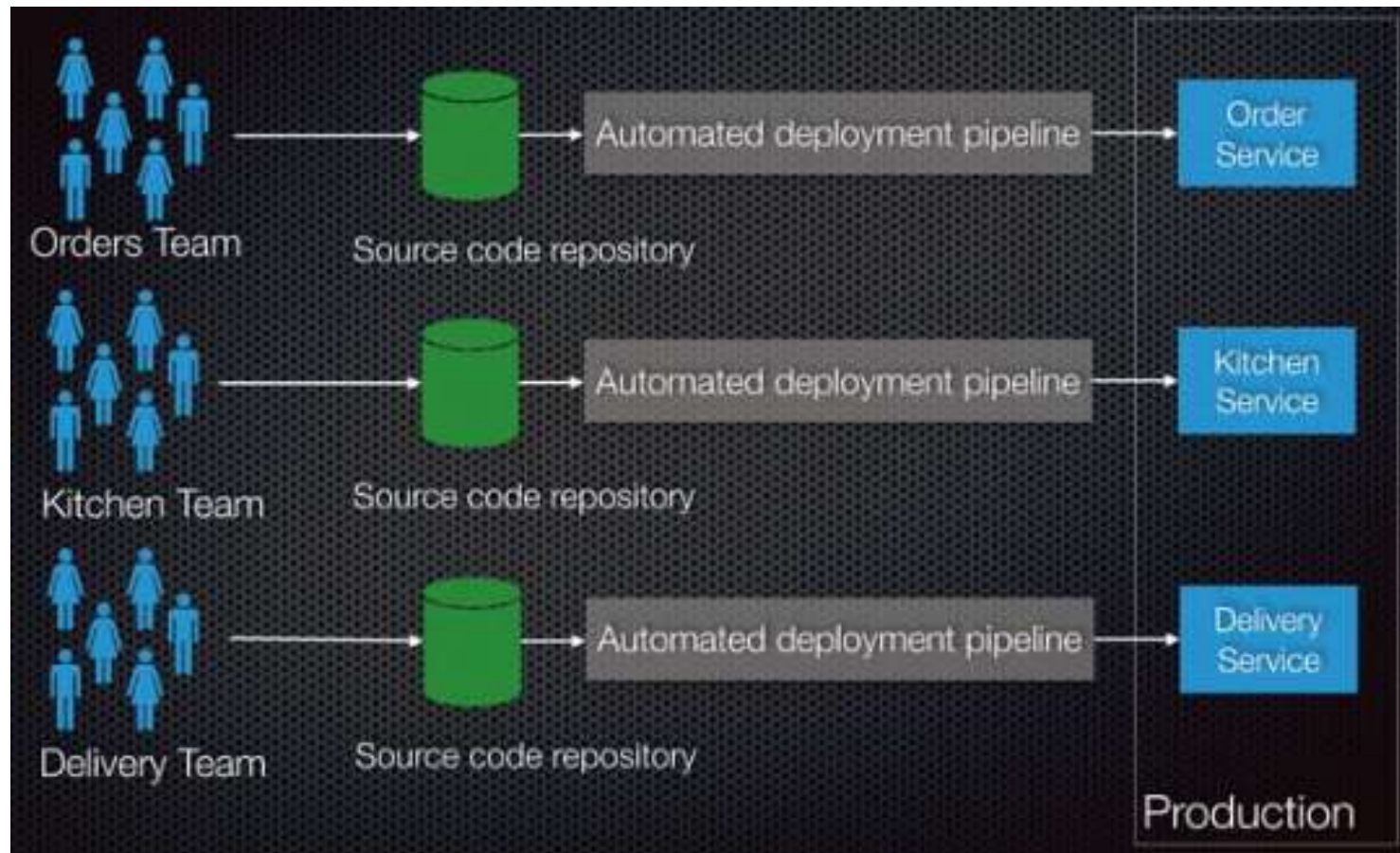
Testing in Microservices

Use DevOps

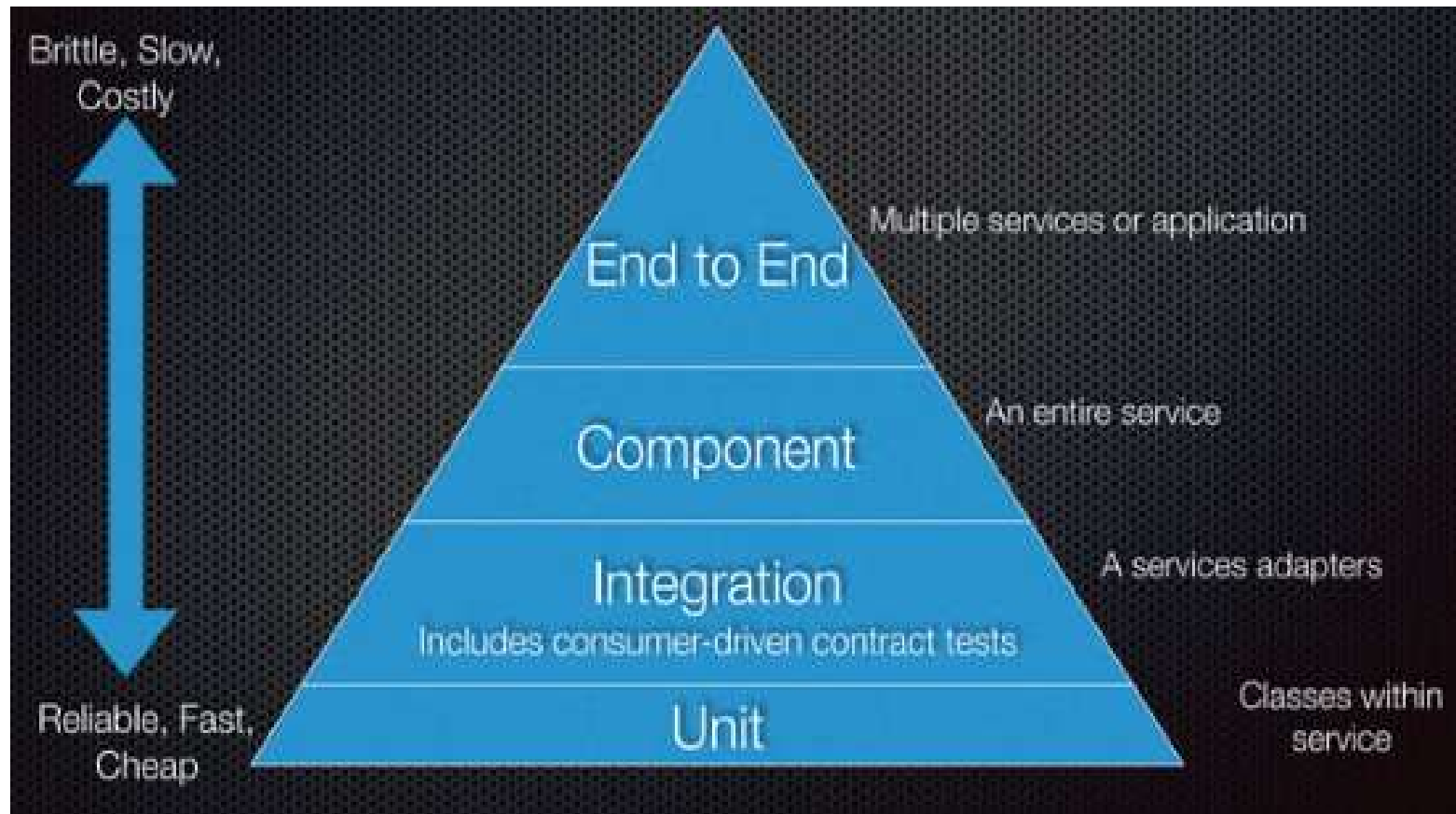


<https://microservices.io/testing/>

Autonomous service teams



Testing Pyramid and Microservices



Testing strategies for Microservice architectures



- Unit Testing
- Component Testing
- Integration Testing
- Contract Testing
- End-to-End testing
- Fault Injection Testing
- **Monitoring and Observability**

Unit Tests



- The size of the unit under test is not defined anywhere some firms, consider writing them at a class level or at a group of related classes.
- Consider keeping the testing units as small as possible. It becomes easier to express the behavior if the test is small since the branching complexity of the unit is lower.

Types of Unit Test



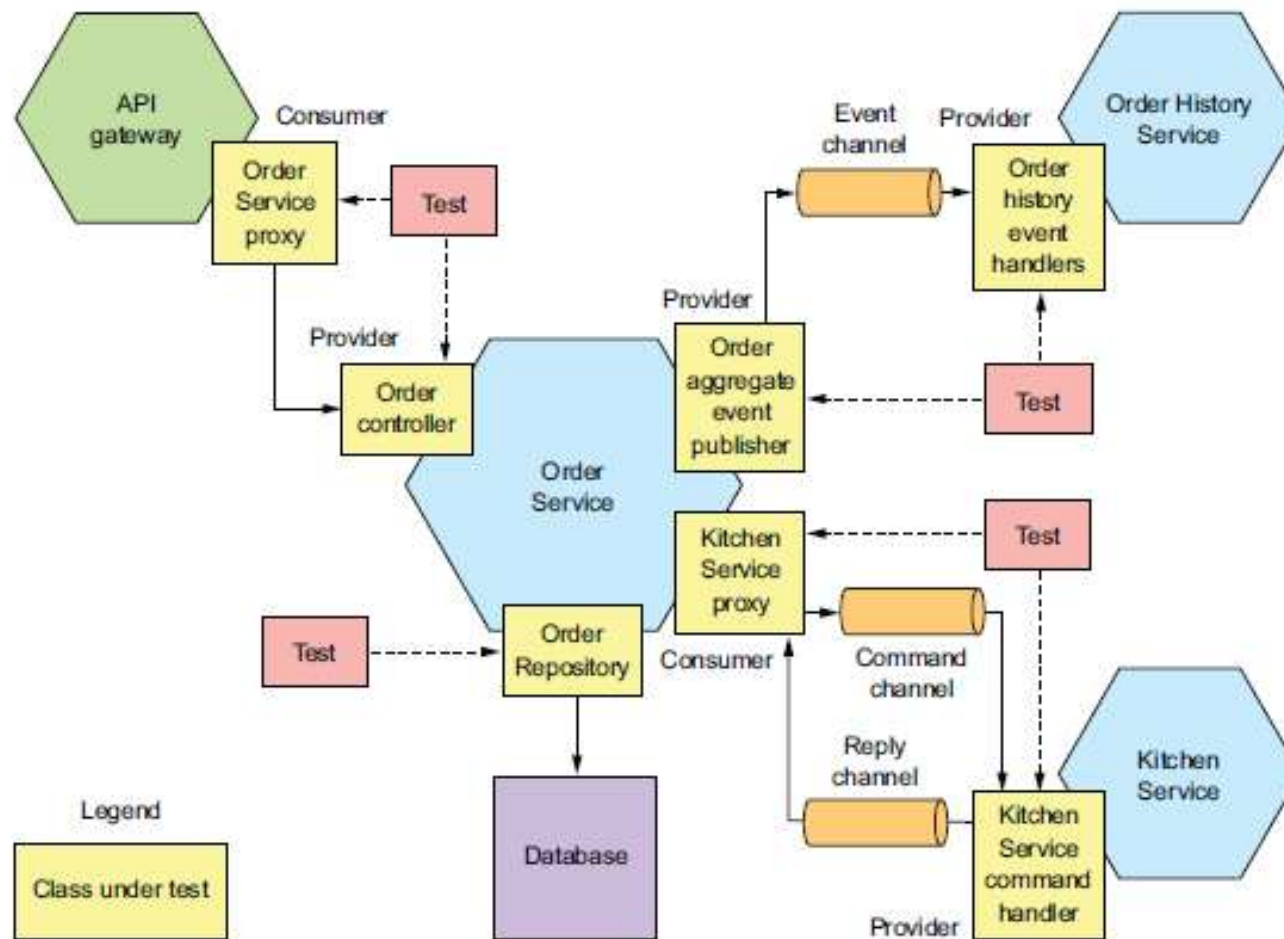
- *Solitary unit* — looks at the interactions and collaborations between an object and its dependencies, which are replaced by the test doubles.
- *Sociable unit test*— focuses on testing the behavior of the module by observing changes in their state.

Component Testing



- A component or microservice is a well-defined coherent and independently replaceable part of a larger system. Once we execute unit tests of all the functions within microservices, it's time to test microservice itself in isolation.
- To test a single microservice in isolation, we need to mock the other microservices. Isolating the microservices in this way using test doubles avoids any complex behavior they exert on execution.

Integration Test



Approaches for integration testing



- Service Integration Testing
- Testing Service Contracts
- Mocking External Dependencies
- End-to-End Testing

Consumer Driven Contact testing



Verify that a service and its clients can communicate while testing them in isolation

Contract testing



- For simplifying integration tests that verify interactions between application services is to use contracts

Interaction style	Consumer	Provider	Contract
REST-based, request/response	API Gateway	Order Service	HTTP request and response
Publish/subscribe	Order History Service	Order Service	Domain event
Asynchronous request/response	Order Service	Kitchen Service	Command message and reply message



-
- *Consumer-side contract tests*: These are tests for the consumer's adapter. They use the contracts to configure stubs that simulate the provider, enabling you to write integration tests for a consumer that don't require a running provider.
 - *Provider-side contract tests*: These are tests for the provider's adapter. They use the contracts to test the adapters using mocks for the adapters's dependencies.

End-to-End Testing



- We usually treat the system as a black box while performing end-to-end tests.
- As Microservice includes more moving parts for the same behavior, it provides coverage of the gaps between the services.



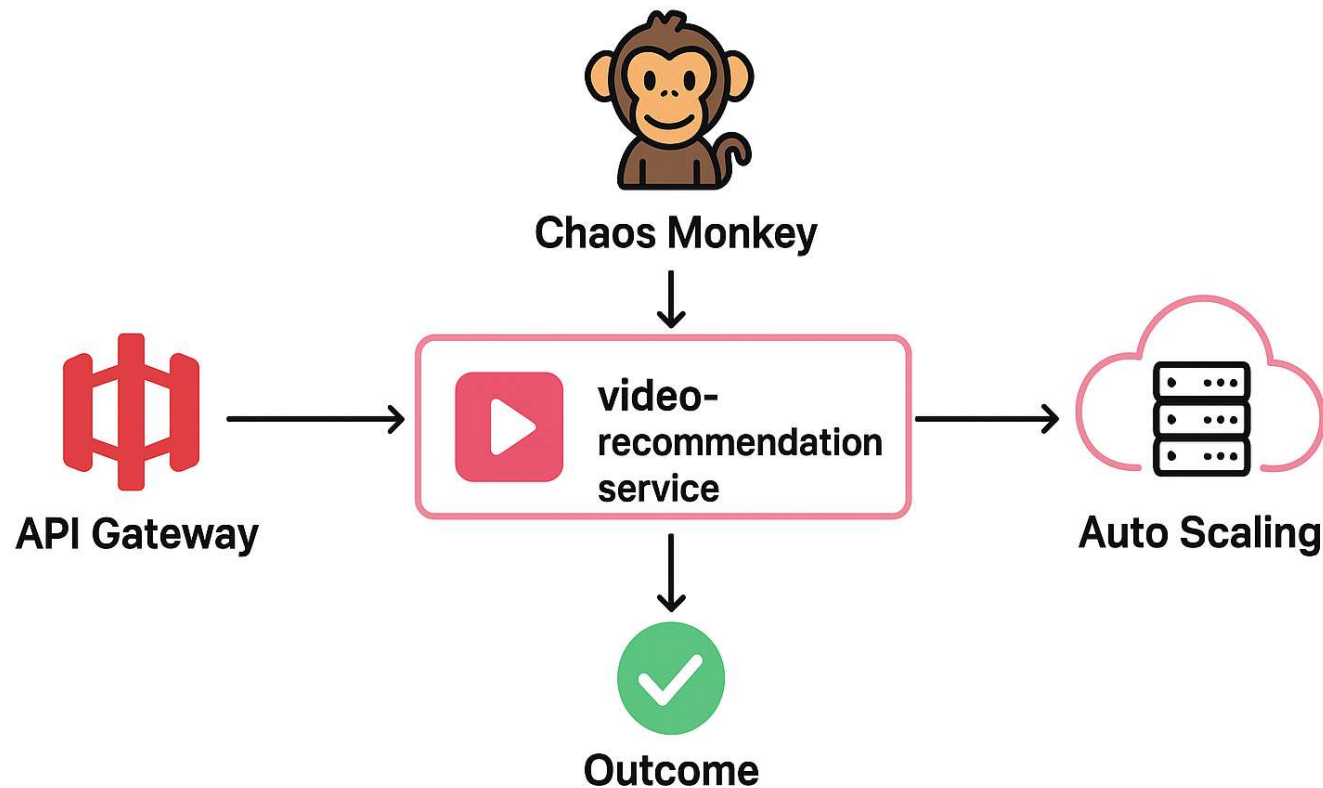
There are various best practices to adhere to while writing efficient E2E tests for microservices:

- Define end-to-end test scenarios that cover critical user journeys or business processes across multiple Microservices.
- To write automated tests that are simple to execute and manage, testing team should make use of testing frameworks and tools.
- Developers should use a continuous integration and delivery (CI/CD) strategy, in which the tests are incorporated into the development cycle and executed automatically anytime the application is changed or released.

Fault injection testing

- Fault injection testing, also known as chaos testing or failure testing, is a technique used to validate the resilience and fault tolerance of systems by deliberately introducing faults or failures into the system and observing how it responds.

Chaos Engineering At Netflix



Tools for Unit testing



- JUnit (for Java), Pytest (for Python), JUnit (for .NET): These are popular unit testing frameworks for writing and executing unit tests for individual components, functions, or classes within microservices.
- Mockito, Mockito.NET: These mocking frameworks help simulate dependencies and external systems during unit testing by creating mock objects or stubs.

Tools for Integration Testing



- **Postman:** Postman is a popular API testing tool that allows you to create and execute tests for RESTful APIs. It supports automated testing, scripting, and validation of API responses.
- **SoapUI:** SoapUI is another API testing tool that specializes in testing SOAP and RESTful web services. It provides features for functional testing, security testing, and performance testing of APIs.
- **WireMock:** WireMock is a lightweight HTTP server for stubbing and mocking HTTP responses. It is commonly used for simulating dependencies and external services during integration testing.

Tools for End-to-End Testing



- **Selenium:** Selenium is a widely used tool for automating web browser interactions and testing web applications. It allows you to create automated end-to-end tests that simulate user interactions across multiple microservices.
- **Cypress:** Cypress is a modern testing tool for web applications that provides an elegant API for writing end-to-end tests. It offers features such as real-time testing, automatic waiting, and debugging capabilities.

Latest Trends in Testing Microservices



- Increased Usage of Contract Testing Frameworks (e.g., Pact)
- Integration of Chaos Engineering in CI/CD for Resilience Validation
- Automated Testing Frameworks with AI-Enhanced Test Generation

Self Study



- <https://docs.github.com/en/code-security/getting-started/securing-your-repository>
- OAuth in detail from textbook
- Testing in detail from textbook
- <https://www.neotys.com/insights/microservices-load-testing>
- <https://netflix.github.io/chaosmonkey/>

References



- Chapter 9,10,11, Microservices Patterns by Chris Richardson
- Chapter 6.4.3, Microservices in Action
- <https://www.ncsc.gov.uk/collection/developers-collection/principles/protect-your-code-repository>
- <https://performancelabus.com/load-testing-for-microservices/>
- <https://netflixtechblog.com/from-chaos-to-control-testing-the-resiliency-of-netflixs-content-discovery-platform-ce5566aef0a4>
- <https://spectrum.ieee.org/chaos-engineering-saved-your-netflix>
- <https://microservices.io/testing/>
- <https://netflixtechblog.com/fit-failure-injection-testing-35d8e2a9bb2>